AgentPulse

A self-hostable observability and evaluation system for multi-agent LLM pipelines. Technical deep dive: architecture, execution flow, evaluation method, measured results, and known limits.

Component	Stack	Size
Backend API and worker	Python, FastAPI, SQLModel, SQLite (WAL), Alembic	`6,625 lines`
Evaluation models	MiniLM-L6-v2 embeddings, DeBERTa-v3-small NLI, ONNX Runtime	`2 local models`
Client SDK	Python, aiohttp, batched async transport	`1,528 lines`
Dashboard	React 19, TypeScript, Vite 6, Tailwind 4, three.js	`8,854 lines`
Test suite	pytest, pytest-asyncio	`36 files / 422 tests`

Line counts exclude generated files and dependencies. Figures throughout this document are read from the repository or measured on a running instance; where a number is an example rather than a measurement, it is labelled as such.

1. The problem

Conventional application monitoring answers whether a service responded. For an LLM agent that is the wrong question: the request can return HTTP 200, in fluent prose, on time, and still be wrong. The failure is in the content, not the transport.

Multi-agent pipelines make this sharper in three ways.

- **Errors compound silently.** A retriever returns a weak document, the synthesiser treats it as established, the writer cites it as settled. Each step is individually plausible; only the chain is wrong.
- **The failure has no stack trace.** Nothing raises. There is no line number to attach a breakpoint to, so post-hoc debugging has nothing to anchor on.
- **Judging with another LLM is expensive.** The common answer, LLM-as-a-judge, bills per token and adds seconds per span. At full coverage on a busy pipeline that cost is prohibitive, so teams sample - and sampling is exactly how a rare hallucination goes unseen.

AgentPulse takes the opposite trade: run small discriminative models locally on CPU, cheaply enough to evaluate every span rather than a sample, and accept narrower coverage than a general judge in exchange.

2. What the system does

An instrumented agent pipeline emits *spans*. A span is one unit of agent work: a planning step, a tool call, a synthesis pass. Each carries its inputs, outputs, timing, and, where relevant, the tool it called and what that tool returned.

AgentPulse accepts those spans, queues them, and evaluates each one asynchronously against four signals. Results are written back as scores and, past configured thresholds, as alerts. A dashboard reads the same REST API a user would.

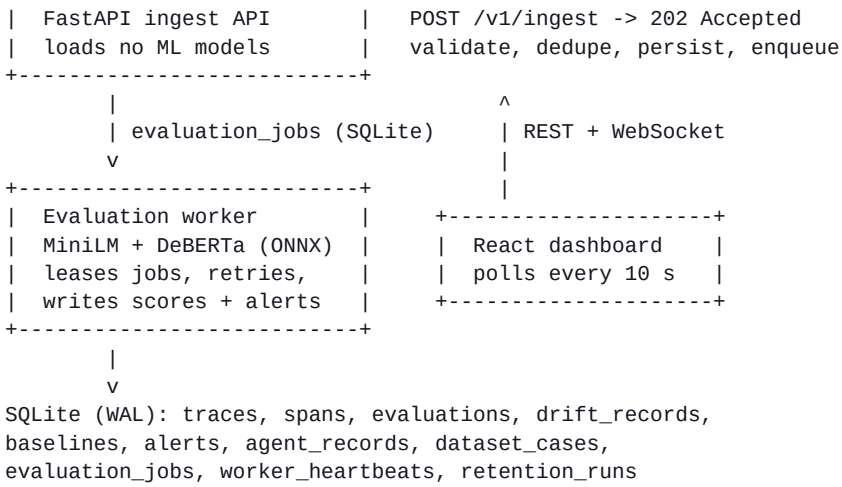
Signal	Question it answers	Method	Maturity
Grounding	Is the output supported by the input evidence?	DeBERTa-v3 NLI: contradiction + 0.3 x neutral (MiniLM similarity kept beside it)	Beta
Drift / ASI	Has this agent's behaviour shifted from its own baseline?	Embedding centroid distance, EMA and windowed	Beta
Disagreement	Do two agents in one trace contradict each other?	Pairwise NLI between agent outputs	Experimental
Tool-claim	Does the narrated claim match what the tool returned?	Regex claim extraction vs recorded tool result	Experimental

Maturity tiers are the project's own. Section 12 states what each tier means in practice, including a defect that currently disables the tool-claim signal end to end.

3. Architecture

Four processes, deliberately separated so that a slow or crashed evaluator cannot stall ingestion.

```
Your agent pipeline
(LangGraph / CrewAI / LangChain / plain Python)
        |
        |  agentpulse SDK: buffers spans, flushes in batches,
        |  fails open if the collector is unreachable
        v
+--------------------------+
```



Why the API holds no models

Loading the evaluator into the API process cost roughly 1.24 GB resident per process and about twenty seconds of startup, for a capability no API route used - every route reads stored evaluation columns rather than running inference. The models now live only in the worker. The behaviour remains one environment variable away (`AGENTPULSE_API_LOAD_MODELS=true`) rather than being deleted outright.

Why evaluation is asynchronous

Ingestion returns `202 Accepted` as soon as the spans are durably written and jobs are enqueued. The caller is never blocked on a 200-millisecond NLI pass. The cost of that choice is that scores appear a short time after the span, which the dashboard has to represent honestly rather than hiding.

4. End-to-end flow

#	Stage	What happens	Where
1	Instrument	A decorator or adapter wraps an agent function and captures inputs, outputs, timing, tool name and tool result.	`sdk/decorators.py`
2	Redact	Privacy filter applies regex redaction and key exclusion before anything leaves the process.	`sdk/privacy.py`
3	Buffer	Spans go into an in-memory buffer, flushed on batch size or interval; failures are swallowed so telemetry cannot break the pipeline.	`sdk/transport.py`
4	Ingest	POST /v1/ingest validates, upserts the trace, writes spans, and enqueues one evaluation job per span.	`routers/ingest.py`
5	Lease	A worker atomically claims a queued job with a 120-second lease and an attempt counter.	`services/job_queue.py`
6	Evaluate	Grounding, tool-claim, disagreement and drift run; the risk score is the weighted mean of grounding and tool-claim (disagreement is stored, not summed).	`services/evaluator.py`
7	Persist	Evaluation and drift rows are written idempotently, keyed on span id.	`services/evaluation_runner.py`
8	Alert	Threshold rules fire, subject to cooldown and hourly caps.	`services/alerting.py`
9	Serve	REST endpoints and a WebSocket expose the results.	`routers/__init__.py`

5. The SDK

The client is deliberately small. Its guiding rule is that observability must never take down the thing it observes.

```
from agentpulse import AgentPulse

pulse = AgentPulse(
    endpoint="http://localhost:8000",
    api_key="your-api-key",
    service_name="production-swarms",
)

@pulse.monitor(agent_id="sql_synthesizer", role="synthesizer")
def run_agent_pipeline(user_query: str):
    schema = fetch_warehouse_schema()
    sql = generate_sql(user_query, schema)
    return validate_and_execute(sql)
```

Properties

- **Fail-open.** `fail_open=True` by default: a collector that is down, slow or misconfigured is swallowed, not raised into the agent.
- **Batched.** Spans buffer and flush by size or interval over aiohttp, so per-span HTTP overhead is amortised.
- **Opt-in payloads.** `capture_inputs` and `capture_outputs` default to `False`. Without them the span carries structure and hashes but no prompt text.
- **Redaction before transport.** The privacy filter runs in the agent's own process, so redacted values never cross the wire.
- **Sampling.** `sampling_rate` exists for pipelines that do not want full coverage, though full coverage is the design intent.

Framework adapters: `instrument_graph()` for LangGraph, `CrewAIAdapter` for CrewAI, `LangChainAdapter` for LangChain. These are explicit wrappers, not automatic monkey-patching, and there is no JavaScript SDK.

6. Ingest and the durable queue

Ingestion and evaluation are separated by a queue table in the same database, which is what makes exactly-once recovery possible without a broker.

State	Meaning
`queued`	Written, waiting for a worker.
`running`	Leased by a worker; the lease carries an expiry.
`succeeded`	Evaluated and persisted.
`failed`	Attempt failed; returns to queued while attempts remain.
`dead_letter`	Exhausted max attempts (3); will not be retried.

A worker claims a job atomically and holds a 120-second lease. If the process dies mid-evaluation the lease expires and the job is reclaimed. Persistence is keyed on span id and is a no-op if an evaluation already exists, so a job replayed after a crash cannot double-write. This path is covered by a crash-recovery test that kills a worker mid-evaluation and asserts the result is written exactly once.

> Dead-letter jobs are terminal by design. On the development instance used for this document, 41 of 121 jobs sit in dead_letter from an earlier misconfigured run - a real operational signal the dashboard surfaces rather than hides.

7. The evaluation cascade

Grounding asks whether an agent's output is supported by the evidence it was given. It runs in two stages, and both run on every span.

```
span (input_summary, output_summary)
      |
      v
Stage 1 - MiniLM-L6-v2 embeddings, cosine similarity   ~19 ms
      |   (no threshold: Stage 2 always runs)
      v
Stage 2 - DeBERTa-v3-small cross-encoder NLI           ~132 ms
      |
      +--> entailment / contradiction / neutral
           -> grounding_risk = contradiction + 0.3 x neutral
```

Both models run locally on CPU. DeBERTa runs on ONNX Runtime when it loads and on PyTorch otherwise; MiniLM always runs on PyTorch. No prompt text leaves the host and no per-token cost is incurred.

Operating point

Thresholds were selected on a 21-case development split and applied unchanged to a held-out 30-case test split: NLI contradiction threshold 0.5, semantic low-similarity floor 0.1, Stage 1 safe threshold 0.85.

8. Measured results

Ablation on dataset v1.0_test, 30 held-out cases.

Configuration	Precision	Recall	F1	Latency
MiniLM only	0.733	0.846	0.786	19.1 ms
DeBERTa NLI only	0.929	1.000	0.963	132.1 ms
Cascade (shipped)	0.929	1.000	0.963	151.2 ms
Full system (all signals)	0.812	1.000	0.897	165.2 ms

The full-system row is the corrected-label re-score at the 0.5 neutral weight that run used (GROUNDING_RECALIBRATION_2026-10.md, T8); the result file recorded 0.963, computed while DeBERTa's entailment and neutral outputs were swapped. Five of the 30 test cases copy dev cases. The shipped score (contradiction + 0.3 x neutral) gives the same 0.963 on the test split at both cut-offs, and F1 0.927 / 0.952 with FPR 0.074 on the 47 truly held-out cases (T3c, T3d).

Confusion matrix at the shipped operating point: 13 true positives, 1 false positive, 0 false negatives, 16 true negatives. False positive rate 0.059, false negative rate 0.000.

> Read this table honestly. On this benchmark the cascade is no more accurate than DeBERTa alone, and is about 19 ms slower, because MiniLM runs on every span: there is no threshold between the stages, so it saves no time on any traffic. The latencies are per-case means from one run on 2026-09-18 (PyTorch path); the 2026-08-23 run of the same script gave 27.8, 188.1 and 215.9 ms. The similarity is kept because the drift signal uses it.

Two further findings from the same run are worth stating plainly: adding the disagreement signal produced results identical to NLI alone, and adding drift *reduced* precision to 0.448. Neither signal improves grounding detection on this dataset. They are reported as separate diagnostics rather than folded into the grounding claim.

Thirty test cases is a small sample. These figures establish that the pipeline works end to end at a chosen operating point; they do not establish generalisation to production traffic.

9. The four signals in detail

9.1 Grounding (Beta)

As described in section 7. Produces `grounding_score` in [0, 1] where higher means less supported, plus the stage that decided it.

9.2 Drift and the Agent Stability Index (Beta)

The drift service tracks two distinct quantities that are easy to confuse.

Metric	What it compares	Used for
`centroid_distance`	One output against the agent's EMA centroid.	Per-span spike. Noisy: on 500 real sessions it flagged 91.7 percent, so it does **not** drive alerts.
`window_centroid_distance`	Mean of a 12-sample current window against the mean of a 20-sample baseline pool.	The sustained signal DRIFT_DETECTED actually fires on, at threshold 0.300.

An agent therefore needs 20 baseline samples plus 12 window samples - 32 evaluated spans - before a sustained drift value exists at all. Until then the field is null, and the dashboard shows it as null rather than as zero.

The Agent Stability Index condenses the drift signals into one 0-100 score:

```
ASI = 100 * sum(w_i * s_i) / sum(w_i)

  s = max(0, 1 - centroid_distance)        w = 0.35
  s = max(0, 1 - 2 * quality_drift)        w = 0.30
  s = max(0, 1 - 5 * error_rate_delta)     w = 0.20
  s = max(0, 1 - tool_drift)               w = 0.15

Terms are included only when that signal exists; weights renormalise
over whichever terms are present.
```

ASI is an explainable heuristic, not a calibrated probability.

9.3 Inter-agent disagreement (Experimental)

Compares assertions between agent outputs within the same trace using the same NLI model, and keeps the worst contradicting pair. Benchmarked on a constructed multi-agent set at threshold 0.6 rather than on collected production traces, which is why it is tiered Experimental.

9.4 Tool-claim assertion (Experimental)

A deterministic check with no model cost: regex patterns extract claims about tool usage from prose, and those are compared against what the trace recorded the tool returning. It catches an agent narrating “retrieved 10 papers” when the tool returned one.

For most of the project's life this signal could not fire at all. The count comparison returns early unless the tool call carries a numeric `result_count`, and the runner built its tool call records with only `tool_name` and `result_summary`. Called directly with a count supplied the detector scored a mismatch at 1.0; through the API it scored 0.0, and across 1,328 evaluations no span had ever scored above zero.

The runner now derives `result_count` from the tool's own result summary, using the same patterns that read a count out of an agent's prose. Verified end to end: an agent claiming ten papers against a tool reporting three

scores 1.0 and raises TOOL_CLAIM_MISMATCH, while a matching claim scores 0.0 and raises nothing.

> The detector's own patterns are unchanged, so its documented limits still hold: a claim must appear in prose with the count adjacent to the noun, and harnesses that emit structured tool_call fields without narrating them are still not covered.

10. Risk aggregation and alerting

Signals combine into one composite risk score by weighted mean over whichever signals produced a value:

Signal	Weight
Grounding	`0.40`
Tool-claim	`0.25`
Disagreement	`0.20`
Semantic	`0.15`

Alert rules are threshold comparisons over stored fields:

Alert	Field	Threshold	Severity
HIGH_HALLUCINATION_RISK	`risk_score`	`> 0.7`	HIGH
GROUNDING_FAILURE	`grounding_score`	`> 0.7`	HIGH
TOOL_CLAIM_MISMATCH	`tool_claim_score`	`> 0.3`	HIGH
DRIFT_DETECTED	`window_centroid_distance`	`> 0.3`	MEDIUM
ASI_DROP	`stability_index`	`< 50`	MEDIUM
ERROR_RATE_SPIKE	`error_rate_delta`	-	HIGH
AGENT_DISAGREEMENT	`disagreement_score`	-	-

Alert storms are bounded by a 900-second per-rule cooldown and a cap of 50 alerts per hour.

11. Storage

SQLite in WAL mode, accessed asynchronously through SQLModel, with schema changes managed by Alembic. Eleven tables:

Table	Holds
`traces`	One row per trace: window, status, span count, overall risk.
`spans`	One row per unit of agent work, with timing and tool fields.
`evaluations`	Per-span scores and the stage that produced them.
`drift_records`	Per-span drift metrics and stability index.
`baselines`	Serialised centroid state: EMA centroids and window baseline pools.
`alerts`	Fired rules with acknowledgement and resolution state.
`agent_records`	Rolling per-agent aggregates: spans, errors, latency, ASI.
`dataset_cases`	Curated benchmark cases promoted from real traces.

Table	Holds
`evaluation_jobs`	The durable queue: status, attempts, lease expiry.
`worker_heartbeats`	Worker liveness, backend in use, jobs processed.
`retention_runs`	Audit trail of what retention deleted and when.

Baseline serialisation matters for correctness, not just convenience. Without it, every restart resets drift state and an agent alerts blindly until its windows refill. Both the EMA centroids and the window baseline pools are persisted and restored on worker start.

12. The dashboard

A React application with two surfaces: a public page that explains the system, and a product console that reads a live instance. The console consumes the same REST API any client would; there is no privileged channel.

View	Reads	Shows
Overview	`/v1/metrics, /v1/agents, /v1/drift, /v1/platform`	Fleet vitals, engine readiness, agent roster
Agents	`/v1/agents, /v1/agents/{id}/health`	Per-agent latency, error rate, risk and drift trend
Traces	`/v1/traces, /v1/traces/{id}`	Trace list, span waterfall, span inspector
Incidents	`/v1/alerts, PATCH /v1/alerts/{id}`	Triage queue, acknowledgement
Drift	`/v1/drift`	Sustained shift, spike, tool drift, baseline pool size
Telemetry Lab	`POST /v1/simulate`	Injects the four backend scenarios
Datasets	`/v1/datasets, POST cases`	Benchmark sets, curation from a span
Experiments	`/v1/experiments`	Recorded experiment runs

Representing absence

The console's data layer maps API responses onto view models and leaves any field the backend does not measure undefined rather than substituting a default. Cost per span, token counts per agent, framework labels and embedding coordinates have no source in the schema, so the UI renders an em-dash. The distinction that matters operationally is between “measured as zero” and “not measured”, and a plausible-looking default erases it.

The same rule governs status. Engine readiness reads `platform.state`; with no worker alive it reads *failing*, not a hardcoded *online*. Agents whose drift windows have not filled read *warming up*, not *stable* - an unmeasured agent is not a healthy agent.

13. Operations

Endpoint	Answers
`/v1/health/live`	Is the process up?
`/v1/health/ready`	Can it serve traffic? (database reachable)
`/v1/health/evaluator`	Is any worker alive? 503 when none, by contract.
`/v1/platform`	Composite state, queue depth by status, worker roster, timing percentiles, reliability counters.

The distinction between *ready* and *evaluator ready* is deliberate: the API can correctly accept spans while nothing is evaluating them. Collapsing the two would report a healthy system that is silently accumulating an unevaluated backlog.

Workers heartbeat and are considered stale after 90 seconds. Retention deletes beyond `retention_days` (default 30) and records what it removed in `retention_runs`. Rate limiting, API-key auth and CORS are handled in middleware; `docker-compose.yml` builds the API and an nginx-served dashboard. It defines no worker service, so a Compose deployment accepts spans but evaluates none until a worker is run against the same database.

14. What the output looks like

Measured on the development instance while writing this document, with one worker running ONNX NLI and PyTorch embeddings.

Table	Rows
traces	20,623
spans	20,771
evaluations	1,328
drift_records	1,328
alerts	98
agent_records	51
dataset_cases	79
evaluation_jobs	121

Evaluation latency with one worker on ONNX NLI (throughput_benchmark.json, 1,000 spans, 2026-08-27): p50 254 ms, p95 417 ms per span, job start to job end. The 151.19 ms ablation figure is a different measurement (PyTorch path, the two grounding models only).

A worked drift detection

To confirm the sustained-drift path end to end, a fresh agent was driven through 20 spans on one topic, then 14 on an unrelated one:

```
after 20 baseline spans:
  baseline_size = 19,  window_centroid_distance = null   (window not filled)

after 14 shifted spans:
  centroid_distance        = 0.3246   (per-span spike)
  window_centroid_distance = 0.9294   (sustained)
  stability_index          = 51.3     (was 100.0)
```

```
-> DRIFT_DETECTED raised on 0.934, not on the 0.32 spike
```

This is the first time in the instance's history that `window_centroid_distance` produced a value: it requires 32 evaluated spans for a single agent, and the in-memory current window does not survive a worker restart. Prior DRIFT_DETECTED rows in the database are legacy spike-based alerts from before the alerting field was switched.

15. Testing

422 tests pass across 36 files (pytest tests/, run on 25 October 2026), covering the durable queue and crash recovery, migrations, retention, health and readiness contracts, resilience, security fixes, self-monitoring, the SDK, the evaluation services, claim extraction, disagreement, dataset curation, LLM adapters, reasoning strategies, and end-to-end LangGraph and real-workflow runs.

> The suite covers Python only. The dashboard has no test framework configured, so its correctness rests on type checking, a successful build, and manual verification against a live backend.

16. Honest limits

Stated plainly, because a monitoring tool that overstates its own coverage is worse than one that admits gaps.

Limit	Detail
Tool-claim needs narrated claims	The count must appear in prose next to the noun. Harnesses that emit structured tool_call fields without narrating them produce no claim to check.
Evaluation coverage is partial	1,328 of 20,771 spans carry an evaluation on the development instance, roughly 6 percent. Most spans predate the current worker or were load-test fill.
Drift needs 32 samples per agent	20 baseline plus 12 window. Short-lived agents never produce a sustained drift value.
Current drift window is not persisted	Baseline pools survive restart; the 12-sample current window does not, so each restart delays sustained detection.
Benchmark is small	30 held-out cases. Sufficient to demonstrate the pipeline, not to claim generalisation.
Disagreement and drift do not help grounding	On the ablation set, disagreement was identical to NLI alone and drift reduced precision to 0.448.
Not OpenTelemetry	The span schema is custom. There is no OTel dependency, import or semantic-convention mapping.
Single-node storage	SQLite with WAL. Appropriate for self-hosted single-instance use; not a multi-writer deployment.
No licence	The repository ships no LICENSE file, so it is source-available rather than open source.

17. Running it

```
# 1. Backend API (no models loaded)
uvicorn app.main:app --app-dir backend --host 127.0.0.1 --port 8000

# 2. Evaluation worker (loads MiniLM + DeBERTa from ./models)
python -m app.worker

# 3. Dashboard
cd dashboard && npm install && npm run dev

# API + dashboard only (compose defines no worker service)
docker compose up
```

Configuration is environment-driven; the defaults that matter:

Variable	Default
AGENTPULSE_DATABASE_URL	sqlite+aiosqlite:///./data/agentpulse.db
AGENTPULSE_API_KEY	change-me-to-a-secure-key
AGENTPULSE_NLI_MODEL	cross-encoder/nli-deberta-v3-small
AGENTPULSE_EMBEDDING_MODEL	sentence-transformers/all-MiniLM-L6-v2
AGENTPULSE_DRIFT_THRESHOLD	0.3
AGENTPULSE_RETENTION_DAYS	30
AGENTPULSE_API_LOAD_MODELS	false

Set HF_HUB_OFFLINE=1 when the host has no network: both models are cached under ./models and the worker will otherwise spend minutes retrying huggingface.co before falling back to PyTorch.